CENG 111 Algoritmalar ve Programlama 5: Program Kontrolü

Öğr.Gör. Şevket Umut ÇAKIR

Pamukkale Üniversitesi

Hafta 5

Anahat

1. Giriş
2. Döngü Temelleri
3. Sayaç Kontrollü Döngü
4. for Döngüsü
5. for Döngü Örnekleri
6. switch Çoklu Seçim Komutu
7. do...while Döngüsü
8. break ve continue Komutları
9. Mantıksal Operatörler
10. Eşitlik ve Atama Operatörlerinin Karıştırılması

Giriş

- Bu derse kadar basit bir C programını okumak ve yazmak konusunda sorun yaşamamalısınız
- Bu ünitede aşağıdakileri öğreneceğiz:
  - `do...while` döngüsünü
  - `switch` çoklu seçme komutunu
  - Döngülerden çıkmak için `break` komutunu
  - Döngüdeki kalan adımları göz ardı etmek için `continue` komutunu

Döngü Temelleri

- Bir çok program tekrarlı işlemler(döngü) içerir
- **Döngü**, **devam etme koşulu** doğru olduğu sürece tekrarlı olarak çalıştırılan bir grup emirdir
- İki farklı döngüden bahsettik:
  - Sayaç kontrollü döngü: kontrol(döngü) değişkeni ile döngünün kaç defa çalışacağı kontrol edilir
  - Gözcü değer kontrollü döngü: Adım sayısı bilinmeyen döngüler için, gözcü değer görülene kadar çalışır

Sayaç Kontrollü Döngü

- Sayaç kontrollü döngü aşağıdakilere ihtiyaç duyar
  - Kontrol değşikeninin **adı**
  - Kontrol değişkeninin **bşlangıç değeri**
  - **Artım** (ya da **azalma**) miktarı
  - Döngü devam etme koşulu, kontrol değişkeninin alacağı **son değeri** belirler

Sayaç Kontrollü Döngü

```c
// kod4_1.c
// Sayaç kontrollü döngü
#include <stdio.h>

int main(void) {
    int sayac = 1; // ilk değer ataması

    while (sayac <= 5) { // döngü koşulu
        printf("%d ", sayac);
        ++sayac; // artım
    }

    puts("");
}
```

```
1 2 3 4 5
```

Sayaç Kontrollü Döngü

- İlk değer ataması: `int sayac = 1;`
- Artım işlemi: `++sayac;`
- Devam koşulu: `sayac <= 5`
- Döngülerdeki kontrol değişkenleri için **tamsayı** kullanın
  - Ondalıklı sayılar(`float`, `double`), hassasiyetlerinden dolayı kesin olmayan değerler alabilirler

for Döngüsü

```
1  // kod4_2.c
2  // for ile sayaç kontrollü döngü
3  #include <stdio.h>
4
5  int main(void) {
6      // ilk değer ataması, döngü koşulu ve artım
7      // for komutu başlığında yer alır
8      for (int sayac = 1; sayac <= 5; ++sayac) {
9          printf("%d ", sayac);
10     }
11
12     puts(""); // yeni satır
13 }
```

```
1 2 3 4 5
```

`for` Döngüsü

- `for` döngüsü sayaç kontrollü döngünün tüm detaylarını ele alır
- Kod çalıştığında, `for` döngüsünde kontrol değişkeni olan `sayac` tanımlanır ve ilk değeri 1 olarak verilir
- Daha sonra döngü devam koşulu(`sayac <= 5`) kontrol edilir, başlangıçta koşul doğru
- Daha sonra for döngüsü `++sayac` ifadesini kullanarak sayacı artırır ve döngü devam koşulunu tekrar kontrol eder
- Bu işlem koşul yanlış olana kadar devam eder

for Döngüsü

- `for (int sayac = 1; sayac <= 5; ++sayac)`

for Döngüsü

- `for (int sayac = 1; sayac <= 5; ++sayac)`



for anahtar sözcüğü

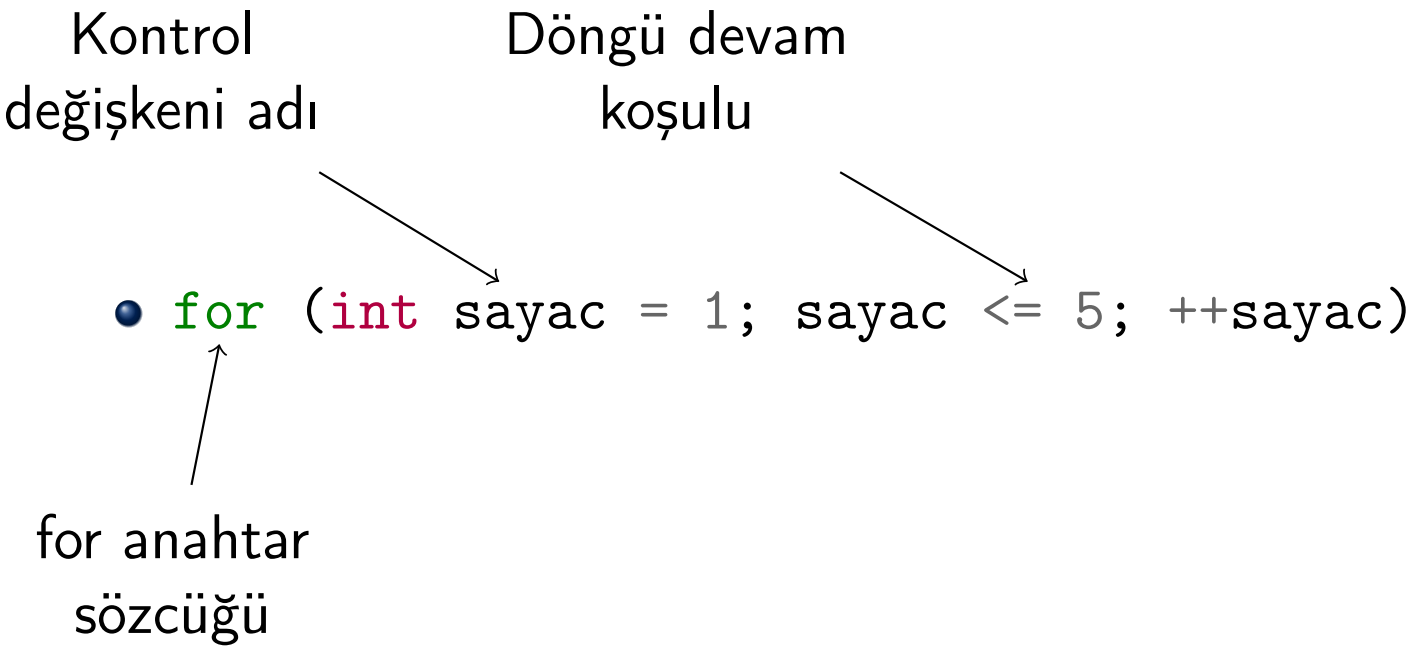
for Döngüsü

Kontrol değişkeni adı

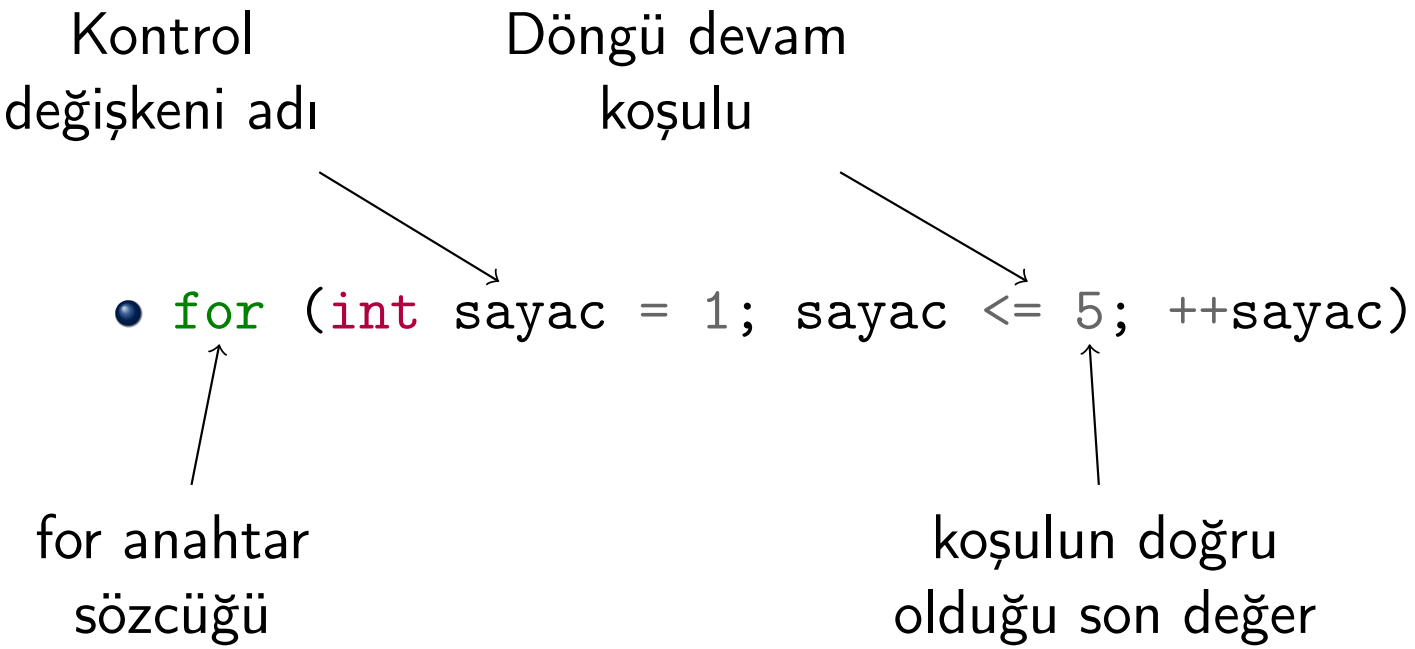
- `for (int sayac = 1; sayac <= 5; ++sayac)`

for anahtar sözcüğü

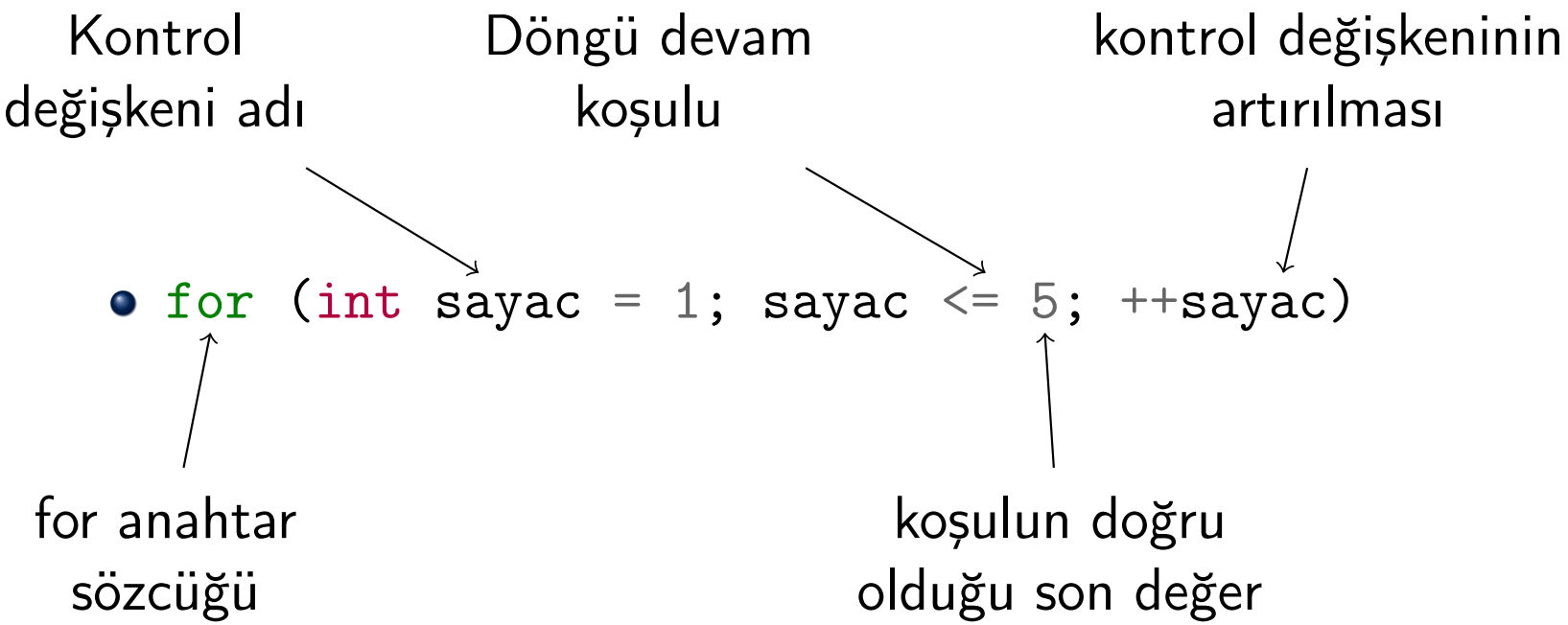
for Döngüsü


Kontrol değişkeni adı
Döngü devam koşulu
for (int sayac = 1; sayac <= 5; ++sayac)
for anahtar sözcüğü

for Döngüsü


Kontrol değişkeni adı
Döngü devam koşulu
for (int sayac = 1; sayac <= 5; ++sayac)
for anahtar sözcüğü
koşulun doğru olduğu son değer

for Döngüsü


Kontrol değişkeni adı
Döngü devam koşulu
kontrol değişkeninin artırılması
for (int sayac = 1; sayac <= 5; ++sayac)
for anahtar sözcüğü
koşulun doğru olduğu son değer

for Döngüsü

- `for` başlığı içinde tanımlanan kontrol değişkeni sadece döngü içinde tanımlıdır
- Döngü koşulu dikkatli ayarlanmalıdır
  - Koşulu `sayac <= 5` yerine `sayac < 5` yazsaydık, döngü dört defa çalışırdı
- `for` döngüsünün genel yapısı aşağıdaki gibidir:

```
for (başlangıç atamaları; döngü devam koşulu; artım) {
        komut(lar)
}
```

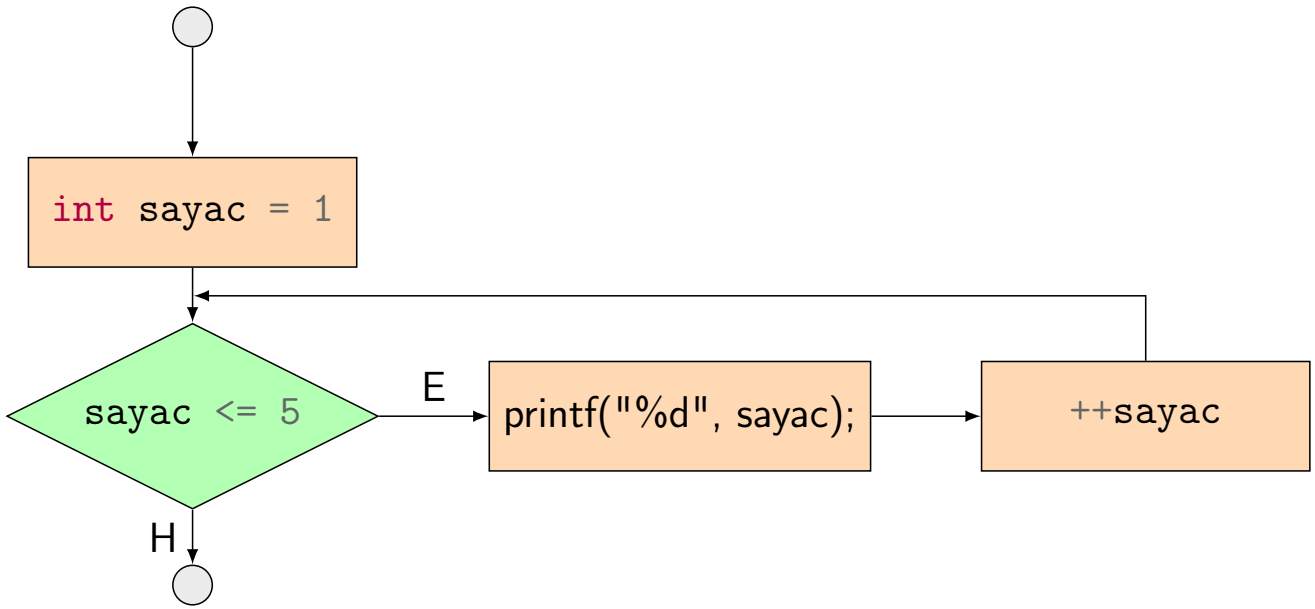
- Koşul hiç yanlış olmazsa, sonsuz döngü oluşur

for Döngüsü

- `for` komutunun başlığındaki ifadeler opsiyoneldir
  - Örneğin devam koşulunu yazmamak, sonsuz döngüye neden olur
    `for (;;);`
- Artım, kendi başına bir komut gibi davranır `sayac = sayac + 1`, `sayac += 1`, `++sayac`, `sayac++`
- Kontrol değişkeni `for` gövdesinde kullanılabilir
  - Kontrol değişkenini `for` içinde değiştirmekten kaçının, genellikle hatalı sonuçlar almanıza neden olur

for Döngüsü

for Döngüsü Akış Diyagramı



for Döngü Örnekleri

- `for (int i = 1; i <= 100; ++i)`

for Döngü Örnekleri

- `for (int i = 1; i <= 100; ++i)` i $\rightarrow$ 1, 2, . . . , 100
- `for (int i = 100; i >= 1; --i)`

for Döngü Örnekleri

- `for (int i = 1; i <= 100; ++i)` i → 1, 2, . . . , 100
- `for (int i = 100; i >= 1; --i)` i → 100, 99, . . . , 1
- `for (int i = 7; i <= 77; i += 7)`

for Döngü Örnekleri

- `for (int i = 1; i <= 100; ++i)` i → 1, 2, . . . , 100
- `for (int i = 100; i >= 1; --i)` i → 100, 99, . . . , 1
- `for (int i = 7; i <= 77; i += 7)` i → 7, 14, . . . , 77
- `for (int i = 20; i >= 2; i -= 2)`

for Döngü Örnekleri

- `for (int i = 1; i <= 100; ++i)` i → 1, 2, . . . , 100
- `for (int i = 100; i >= 1; --i)` i → 100, 99, . . . , 1
- `for (int i = 7; i <= 77; i += 7)` i → 7, 14, . . . , 77
- `for (int i = 20; i >= 2; i -= 2)` i → 20, 18, . . . , 2
- `for (int j = 2; j <= 17; j += 3)`

for Döngü Örnekleri

- `for (int i = 1; i <= 100; ++i)` i → 1, 2, . . . , 100
- `for (int i = 100; i >= 1; --i)` i → 100, 99, . . . , 1
- `for (int i = 7; i <= 77; i += 7)` i → 7, 14, . . . , 77
- `for (int i = 20; i >= 2; i -= 2)` i → 20, 18, . . . , 2
- `for (int j = 2; j <= 17; j += 3)` j → 2, 5, 8, 11, 14, 17
- `for (int j = 44; j >= 0; j -= 11)`

for Döngü Örnekleri

- `for (int i = 1; i <= 100; ++i)` i → 1, 2, . . . , 100
- `for (int i = 100; i >= 1; --i)` i → 100, 99, . . . , 1
- `for (int i = 7; i <= 77; i += 7)` i → 7, 14, . . . , 77
- `for (int i = 20; i >= 2; i -= 2)` i → 20, 18, . . . , 2
- `for (int j = 2; j <= 17; j += 3)` j → 2, 5, 8, 11, 14, 17
- `for (int j = 44; j >= 0; j -= 11)` j → 44, 33, 22, 11, 0

for Döngü Örnekleri

2'den 100'e Çift Sayıların Toplamı

```c
// kod4_3.c
// for ile toplama
#include <stdio.h>

int main(void) {
    int toplam = 0; // toplamın ilk değerini ver

    for (int sayi = 2; sayi <= 100; sayi += 2) {
        toplam += sayi;
    }

    printf("Toplam: %d\n", toplam);
}
```

```
Toplam: 2550
```

Uygulama: Bileşik Faiz Hesaplama

- Bir kişi %5 faiz getiren bir tasarruf hesabına 1000,00 TL yatırıyor. Tüm faizlerin hesapta bırakıldığı varsayılarak, her yılın sonunda 10 yıl boyunca hesaptaki para miktarını hesaplayın ve yazdırın. Bu miktarları belirlemek için aşağıdaki formülü kullanın:

  $a = p(1 + r)^n$

- p, başlangıçta yatırılan miktar (yani anapara, burada 1000,00 TL),
- r, yıllık faiz oranı (örneğin, %5 için 0,05),
- n, yıl sayısı (burada 10) ve
- a, n yılının sonunda hesaptaki miktardır.

Uygulama: Bileşik Faiz Hesaplama

```c
// kod4_4.c
// Bileşik faiz hesaplama
#include <stdio.h>
#include <math.h>

int main(void) {
    double anapara = 1000.0; // başlangıç anapara
    double oran = 0.05; // yıllık faiz oranı

    //Çıktı tablosu başlıkları
    printf("%4s%21s\n", "Yil", "Bankadaki miktar");

    // 10 yıl için bankadaki miktarı hesapla
    for (int yil = 1; yil <= 10; ++yil) {
        // İlgili yıl için yeni miktarı hesapla
        double miktar = anapara * pow(1.0 + oran, yil);
        // Tabloya bir satır yazdır
        printf("%4d%21.2f\n", yil, miktar);
    }
}
```

Uygulama: Bileşik Faiz Hesaplama

```
Yil     Bankadaki miktar
  1              1050.00
  2              1102.50
  3              1157.63
  4              1215.51
  5              1276.28
  6              1340.10
  7              1407.10
  8              1477.46
  9              1551.33
 10              1628.89
```

Uygulama: Bileşik Faiz Hesaplama

- Üs almak için `pow` fonksiyonuna ihtiyaç duyarız ve `math.h` kütüphanesinden yükleriz
- printf içindeki `"%21.2f"` ifadesi ondalıklı sayının toplam 21 basamak olacağını(nokta ve ondalık kısım dahil) ve virgülden sonra iki basamak yazılacağını belirtir
- `float` türü bellekte 4 bayt alan kaplar, `double` türü ise 8 bayt alan kaplar[1]
  - `double` daha hassas bir veri türüdür
- Ondalıklı sayılar yaklaşık sayılardır, tam olarak temsil edilmeyebilirler
- Yuvarlanan sayılar yanıltıcı olabilir. Örneğin 14.234(14.23'e yuvarlanır) ve 18.673(18.67'ye yuvarlanır) toplanırsa $14.23 + 18.67 = 32.91$ gibi bir sonuç ortaya çıkabilir
- Temsil hassasiyetinden kaynaklı sorunlar olabilir. Örneğin `double d = 123.02;` değerini yazdırmak istediğimizde `123.01999999...` gibi bir sonuçla karşılaşabiliriz

[1] `https://en.cppreference.com/w/c/language/arithmetic_types`

switch Çoklu Seçim Komutu

```c
// kod4_5.c
// switch ile harf notlarını saymak
#include <stdio.h>

int main(void) {
    int aSayisi = 0;
    int bSayisi = 0;
    int cSayisi = 0;
    int dSayisi = 0;
    int fSayisi = 0;

    puts("Harf notlarını girin.");
    puts("Girdiyi sonlandırmak için EOF karakteri girin.");
    int notu = 0; // bir not

    // Kullanıcı end-of-file tuşuna basana kadar dön
    while ((notu = getchar()) != EOF) {
        // hangi notların girildiğini belirle
        switch (notu) { // while içinde switch
        case 'A': // not büyük 'A'
```

switch Çoklu Seçim Komutu

```
21          case 'a': // veya küçük 'a'
22              aSayisi++;
23              break; // switch'den çıkmak için gerekli
24          case 'B': // not büyük 'B'
25          case 'b': // veya küçük 'b'
26              bSayisi++;
27              break; // switch'den çıkmak için gerekli
28          case 'C': // not büyük 'C'
29          case 'c': // veya küçük 'c'
30              cSayisi++;
31              break; // switch'den çıkmak için gerekli
32          case 'D': // not büyük 'D'
33          case 'd': // veya küçük 'd'
34              dSayisi++;
35              break; // switch'den çıkmak için gerekli
36          case 'F': // not büyük 'F'
37          case 'f': // veya küçük 'f'
38              fSayisi++;
39              break; // switch'den çıkmak için gerekli
40          case '\n': // yeni satır
```

switch Çoklu Seçim Komutu

```
41          case '\t': // sekme
42          case ' ':  // ve boşluğu göz ardı et
43              break;
44          default: // diğer bütün karakterler
45              printf("%s", "Hatalı not girildi.");
46              puts("Yeni not girin.");
47              break;
48          } // switch sonu
49      } // while sonu
50
51      // Sonuçların özetini yazdır
52      puts("\nHer not için toplamlar: ");
53      printf("A: %d\n", aSayisi);
54      printf("B: %d\n", bSayisi);
55      printf("C: %d\n", cSayisi);
56      printf("D: %d\n", dSayisi);
57      printf("F: %d\n", fSayisi);
58  }
```

switch Çoklu Seçim Komutu

```
Harf notlarını girin.
Girdiyi sonlandırmak için EOF karakteri girin.
a
b
c
C
A
d
f
C
E
Hatalı not girildi.Yeni not girin.
D
A
b

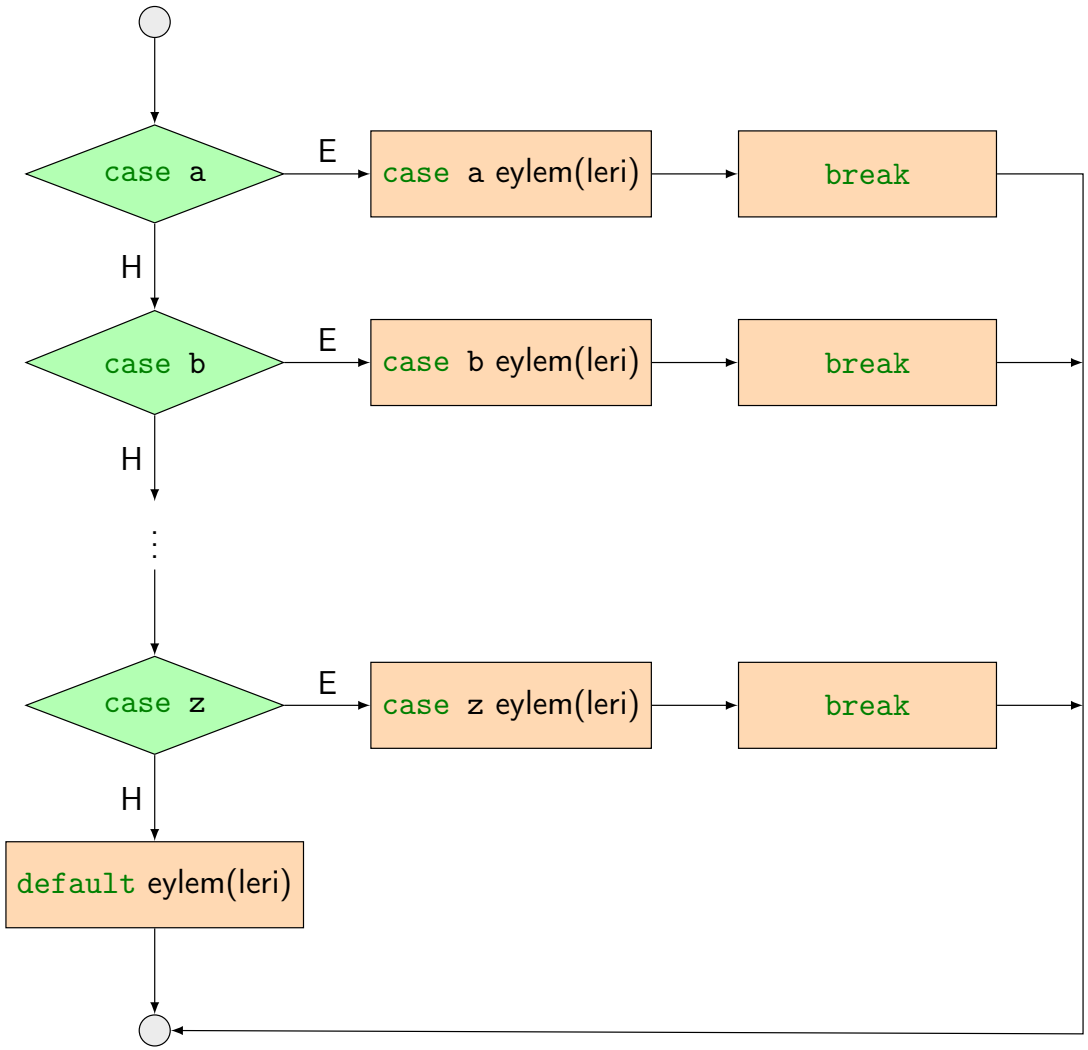
Her not için toplamlar:
A: 3
B: 2
C: 3
D: 2
F: 1
```

switch Çoklu Seçim Komutu

- `while ((notu = getchar()) != EOF)`
  - Klavyeden girilen karakteri `notu` değişkenine atar
  - Atama işleminin bir değeri vardır ve kullanıcının girdiği değerdir
  - Değer `int` türünde olduğu için sayısal değer saklanır
  - Örneğin kullanıcı 'a' karakteri girerse `notu` değişkeninin değeri 97(ASCII karşılığı) olacaktır
- `a = b = c = 0;`
  - Atama işleminin değeri vardır
  - Atama işleminin gruplanması sağdan soladır
  - İlk olarak `c = 0` işlemi yapılır ve bu ifadenin değeri 0 olur
  - Daha sonra sırayla `b = 0` ve `a = 0` işlemleri yapılır
- EOF karakteri
  - Dosya sonu karakteridir(end of file)
  - EOF karakteri gözcü değer olarak kullanılır
  - Linux/Unix/macOS sistemlerde Ctrl + d,
  - Windows sistemlerde Ctrl + z ile yapılır

switch Çoklu Seçim Komutu

switch Çoklu Seçim Komutu Akış Diyagramı



do...while Döngüsü

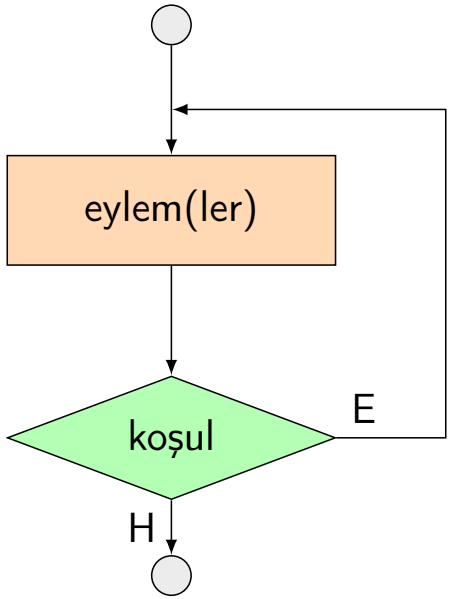
```c
// kod4_6.c
// do...while döngüsünü kullanmak
#include <stdio.h>

int main(void) {
    int sayac = 1;
    do {
        printf("%d ", sayac);
    } while(++sayac <= 5);
}
```

```
1 2 3 4 5
```

do...while Döngüsü

do...while Döngüsü Akış Diyagramı



break ve continue Komutları

```c
// kod4_7.c
// for döngüsünde break ifadesi kullanmak
#include <stdio.h>

int main(void) {
    int x = 1; // burada tanımlandı, for'dan sonra kullanılabilir
    // 10 defa dön
    for (; x <= 10; ++x) {
        // eğer x 5 ise döngüyü sonlandır
        if (x == 5) {
            break;
        }
        printf("%d ", x);
    }
    printf("\nx == %d iken döngüden çıktı\n", x);
}
```

```
1 2 3 4
x == 5 iken döngüden çıktı
```

break ve continue Komutları

```c
// kod4_8.c
// for döngüsünde continue ifadesi kullanmak
#include <stdio.h>

int main(void) {
    // 10 defa dön
    for (int x = 1; x <= 10; ++x) {
        // eğer x 5 ise döngünün diğer adımından devam et
        if (x == 5) {
            continue; // döngü gövdesindeki kalan kodları göz ardı et
        }
        printf("%d ", x);
    }
    puts("\n5 değerinin yazılması göz ardı edildi.");
}
```

```
1 2 3 4 6 7 8 9 10
5 değerinin yazılması göz ardı edildi.
```

Mantıksal Operatörler

- Bu ana kadar görülen koşullar basit koşullardı
  - `sayac <= 10`
  - `toplam > 1000`
  - `notu != -1`
- Her karar tek bir koşula bağlıydı
- Birden fazla koşulu test etmek için mantıksal operatörler bulunur
  - `&&`: Mantıksal **VE**
  - `||`: Mantıksal **VEYA**
  - `!`: Mantıksal **DEĞİL**

Mantıksal VE

```
if (cinsiyet == 1 && yas >= 65) {
        ++yasliKadin;
}
```

Mantıksal VE

```
if (cinsiyet == 1 && yas >= 65) {
        ++yasliKadin;
}
```

- Doğruluk tablosu

ifade1	ifade2	ifade1 && ifade2
0	0	0
0	sıfırdan farklı	0
sıfırdan farklı	0	0
sıfırdan farklı	sıfırdan farklı	1

Mantıksal VEYA

```
if (donemOrtalamasi >= 90 || finalSinavi >= 90) {
        puts("Öğrencinin notu A'dır");
}
```

Mantıksal VEYA

```
if (donemOrtalamasi >= 90 || finalSinavi >= 90) {
        puts("Öğrencinin notu A'dır");
}
```

- Doğruluk tablosu

ifade1	ifade2	ifade1 | ifade2
0	0	0
0	sıfırdan farklı	1
sıfırdan farklı	0	1
sıfırdan farklı	sıfırdan farklı	1

Mantıksal DEĞİL

```
if (!(notu == gozcuDeger)) {
        printf("Sonraki not: %f\n", notu);
}
```

Mantıksal DEĞİL

```
if (!(notu == gozcuDeger)) {
        printf("Sonraki not: %f\n", notu);
}
if (notu != gozcuDeger) {
        printf("Sonraki not: %f\n", notu);
}
```

Mantıksal DEĞİL

```
if (!(notu == gozcuDeger)) {
        printf("Sonraki not: %f\n", notu);
}
if (notu != gozcuDeger) {
        printf("Sonraki not: %f\n", notu);
}
```

- Doğruluk tablosu

ifade	!ifade
0	1
sıfırdan farklı	0

Kısa Devre Değerlendirme(Short-Circuit Evaluation)

- İfadenin tamamını değerlendirmeden bir sonuca varabiliyorsak, kalan ifadeleri değerlendirmeye gerek yoktur

Kısa Devre Değerlendirme(Short-Circuit Evaluation)

- İfadenin tamamını değerlendirmeden bir sonuca varabiliyorsak, kalan ifadeleri değerlendirmeye gerek yoktur

```
if (cinsiyet == 1 && yas >= 65) {
        ++yasliKadin;
}
```

Kısa Devre Değerlendirme(Short-Circuit Evaluation)

- İfadenin tamamını değerlendirmeden bir sonuca varabiliyorsak, kalan ifadeleri değerlendirmeye gerek yoktur

```
if (cinsiyet == 1 && yas >= 65) {
        ++yasliKadin;
}
if (donemOrtalamasi >= 90 || finalSinavi >= 90) {
        puts("Öğrencinin notu A'dır");
}
```

Eşitlik ve Atama Operatörlerinin Karıştırılması

- Aşağıdaki kodu inceleyelim

```
if (odemeKodu == 4) {
        printf("%s", "Bonus kazandınız!");
}
```

Eşitlik ve Atama Operatörlerinin Karıştırılması

- Aşağıdaki kodu inceleyelim

```
if (odemeKodu == 4) {
        printf("%s", "Bonus kazandınız!");
}
```

- Yanlışlıkla aşağıdaki gibi yazalım

```
if (odemeKodu = 4) {
        printf("%s", "Bonus kazandınız!");
}
```

- Kod derlenir mi? Kod çalışır mı?

Eşitlik ve Atama Operatörlerinin Karıştırılması

- Aşağıdaki kodu inceleyelim

```
if (odemeKodu == 4) {
        printf("%s", "Bonus kazandınız!");
}
```

- Yanlışlıkla aşağıdaki gibi yazalım

```
if (odemeKodu = 4) {
        printf("%s", "Bonus kazandınız!");
}
```

- Kod derlenir mi? Kod çalışır mı?
- Karşılaştırmada değişken adını sağa yazmak bu tip hataların önüne geçer. `x == 7` yerine `7 == x`